PLANO DE TESTES: Teste Unitário com Jasmine

Título do Plano de Testes: Teste Unitário com Jasmine

Data de Criação: 27/08/2026

Autor: Mateus Oliveira

Objetivo: Este plano de testes tem como objetivo definir as estratégias, processos e recursos necessários para a realização de testes unitários dos módulos de Calculadora e Validação de Maioridade.

Escopo: O escopo deste plano de testes inclui a verificação das funcionalidades de soma, subtração, multiplicação e divisão do módulo Calculadora, além da validação da regra de maioridade do módulo de Acesso, considerando jogadores com 18 anos ou mais como maiores de idade.

Ambiente de Teste:

- Sistema Operacional: Linux
- Navegador: Google Chrome
- Versão do Sistema: v1.0
- Framework de Teste: Jasmine Standalone

Data de Início: 27/08/2026

Data de Conclusão: 27/08/2026

Equipe de Teste:

- Líder de Teste: Mateus Oliveira
- Engenheiro de Teste: Mateus Oliveira

Tipos de Teste:

- Teste Unitário

- Teste Estrutural (Caixa Branca)

Processo de Teste:

1. Preparação do Ambiente de Teste

2. Preparação dos Casos de Teste

3. Execução dos Casos de Teste

4. Avaliação dos Resultados

Ferramentas de Teste: Jasmine Standalone

Aprovação:

- Aprovação do Líder de Teste: Mateus Oliveira
- Aprovação do Gerente de Projeto: Tutor

Casos de Teste

Caso de Teste 1 – Soma Válida

ID do Caso de Teste: TC-001 – Somas Válidas

Propósito: Verificar se a função de soma realiza corretamente a adição de dois números.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função somar deve estar carregada no ambiente de teste.

Passos de Execução:

1. Executar a função de soma.

2. Informar o primeiro número: 10.

3. Informar o segundo número: 5.

Resultado Esperado:

- A função deve retornar 15.

Código Jasmine:

```
it("deve somar dois números corretamente", function () {
    expect(somar(10, 5)).toBe(15);
});
```

Caso de Teste 2 – Subtração Válida

ID do Caso de Teste: TC-002 – Subtrações Válidas

Propósito: Verificar se a função de subtração realiza corretamente a diferença entre dois números.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função subtrair deve estar carregada no ambiente de teste.

Passos de Execução:

1. Executar a função de subtração.

2. Informar o primeiro número: 10.

3. Informar o segundo número: 5.

Resultado Esperado:

- A função deve retornar 5.

Código Jasmine:

```
it("deve subtrair dois números corretamente", function () {
    expect(subtrair(10, 5)).toBe(5);
});
```

Caso de Teste 3 – Multiplicação Válida

ID do Caso de Teste: TC-003 – Multiplicações Válidas

Propósito: Verificar se a função de multiplicação realiza corretamente o produto de dois números.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função multiplicar deve estar carregada no ambiente de teste.

Passos de Execução:

1. Executar a função de multiplicação.

2. Informar o primeiro número: 10.

3. Informar o segundo número: 5.

Resultado Esperado:

- A função deve retornar 50.

Código Jasmine:

```
it("deve multiplicar dois números corretamente", function () {
    expect(multiplicar(10, 5)).toBe(50);
});
```

Caso de Teste 4 – Divisão Válida

ID do Caso de Teste: TC-004 – Divisões Válidas

Propósito: Verificar se a função de divisão realiza corretamente a divisão de dois números.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função dividir deve estar carregada no ambiente de teste.
- O divisor deve ser diferente de zero.

Passos de Execução:

1. Executar a função de divisão.

2. Informar o primeiro número: 10.

3. Informar o segundo número: 2.

Resultado Esperado:

- A função deve retornar 5.

Código Jasmine:

```
it("deve dividir dois números corretamente", function () {
    expect(dividir(10, 2)).toBe(5);
});
```

Caso de Teste 5 – Divisão por Zero

ID do Caso de Teste: TC-005 – Divisão por Zero

Propósito: Verificar se a função de divisão trata corretamente uma tentativa de divisão por zero.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função dividir deve estar carregada no ambiente de teste.

Passos de Execução:

1. Executar a função de divisão.

2. Informar o primeiro número: 10.

3. Informar o segundo número: 0.

Resultado Esperado:

- A função deve lançar o erro 'Não é possível dividir por zero'.

Código Jasmine:

```
it("deve impedir divisão por zero", function () {
    expect(function () {
        dividir(10, 0);
    }).toThrowError("Não é possível dividir por zero");
});
```

Caso de Teste 6 – Menor de Idade

ID do Caso de Teste: TC-006 – Menor de Idade

Propósito: Verificar se jogadores com idade inferior a 18 anos são identificados como menores de idade.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função verificarMaioridade deve estar carregada no ambiente de teste.

Passos de Execução:

1. Executar a função de validação de idade.

2. Informar a idade: 17.

Resultado Esperado:

- A função deve retornar 'Menor Idade'.

Código Jasmine:

```
it("deve identificar 17 anos como menor de idade", function () {
    expect(verificarMaioridade(17)).toBe("Menor Idade");
});
```

Caso de Teste 7 – Limite de Maioridade

ID do Caso de Teste: TC-007 – Maioridade aos 18 Anos

Propósito: Verificar o comportamento da regra exatamente no limite de 18 anos.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função verificarMaioridade deve estar carregada no ambiente de teste.

Passos de Execução:

1. Executar a função de validação de idade.

2. Informar a idade: 18.

Resultado Esperado:

- A função deve retornar 'Maior Idade'.

Código Jasmine:

```
it("deve identificar 18 anos como maior de idade", function () {
    expect(verificarMaioridade(18)).toBe("Maior Idade");
});
```

Caso de Teste 8 – Maior de Idade

ID do Caso de Teste: TC-008 – Maior de Idade

Propósito: Verificar se jogadores acima de 18 anos são identificados como maiores de idade.

Pré-condições:

- O sistema deve estar ativo e disponível.
- A função verificarMaioridade deve estar carregada no ambiente de teste.

Passos de Execução:

1. Executar a função de validação de idade.

2. Informar a idade: 25.

Resultado Esperado:

- A função deve retornar 'Maior Idade'.

Código Jasmine:

```
it("deve identificar 25 anos como maior de idade", function () {
    expect(verificarMaioridade(25)).toBe("Maior Idade");
});
```

Desenho de Teste Jasmine

O desenho dos testes utiliza a sintaxe básica do Jasmine, composta principalmente por describe, it e expect. O teste estrutural (caixa branca) considera os caminhos internos das funções, incluindo os diferentes resultados possíveis das operações e os dois caminhos da condição de maioridade.

1. Módulo Calculadora

```
describe("Módulo Calculadora", function () {

    describe("Operação de soma", function () {
        it("deve somar dois números corretamente", function () {
            expect(somar(10, 5)).toBe(15);
        });
    });

    describe("Operação de subtração", function () {
        it("deve subtrair dois números corretamente", function () {
            expect(subtrair(10, 5)).toBe(5);
        });
    });

    describe("Operação de multiplicação", function () {
        it("deve multiplicar dois números corretamente", function () {
            expect(multiplicar(10, 5)).toBe(50);
        });
    });

    describe("Operação de divisão", function () {
```

```
    it("deve dividir dois números corretamente", function () {
        expect(dividir(10, 2)).toBe(5);
    });

    it("deve impedir divisão por zero", function () {
        expect(function () {
            dividir(10, 0);
        }).toThrowError("Não é possível dividir por zero");
    });
});
```

2. Módulo de Acesso

```
describe("Módulo de Acesso", function () {

    it("deve identificar 17 anos como menor de idade", function () {
        expect(verificarMaioridade(17)).toBe("Menor Idade");
    });

    it("deve identificar 18 anos como maior de idade", function () {
        expect(verificarMaioridade(18)).toBe("Maior Idade");
    });

    it("deve identificar 25 anos como maior de idade", function () {
        expect(verificarMaioridade(25)).toBe("Maior Idade");
    });
});
```

Código das Unidades Testadas

calculadora.js

```
function somar(a, b) {
    return a + b;
}

function subtrair(a, b) {
    return a - b;
}

function multiplicar(a, b) {
    return a * b;
}

function dividir(a, b) {
    if (b === 0) {
        throw new Error("Não é possível dividir por zero");
    }

    return a / b;
}
```

acesso.js

```
function verificarMaioridade(idade) {
    if (idade >= 18) {
        return "Maior Idade";
    }
    return "Menor Idade";
}
```

Resultados Esperados e Avaliação

Após a execução do SpecRunner.html no navegador, espera-se que todos os casos de teste sejam aprovados pelo Jasmine. Os resultados esperados são 8 casos de teste aprovados e nenhuma falha. A avaliação considera que as funções atendem aos requisitos definidos no plano quando os valores retornados correspondem aos resultados esperados.

ID	Módulo	Resultado Esperado	Situação
TC-001	Calculadora	15	Aprovado
TC-002	Calculadora	5	Aprovado
TC-003	Calculadora	50	Aprovado
TC-004	Calculadora	5	Aprovado
TC-005	Calculadora	Erro de divisão por zero	Aprovado
TC-006	Acesso	Menor Idade	Aprovado
TC-007	Acesso	Maior Idade	Aprovado
TC-008	Acesso	Maior Idade	Aprovado

Capture de Tela do Teste Com Jasmin

